

README

Module Boundary Matrix

Open `index.html`. No build step, no dependency, no network, no server.

This is the interactive half of §5. It answers questions about module boundaries from a corpus of **880 executed evaluations** — 44 languages × 20 properties — and **362 adjudicated bypass records**, every verdict produced by a probe that was built and run rather than read off a manual.

What it answers

Pick the boundaries you need and, optionally, how you need them realised and which facilities your code ships. The output is grouped under the questions it answers:

Has this been built before?	who provides it, whether it is rare or unoccupied, and — if it cannot occur — the precondition that rules it out
What should I read as a reference implementation?	the matching languages ranked by how closely they follow the group's own design, and the ones that reach it by an unusual route
What kind of design is this?	one decision or several; the closest family and the profile that defines it; whether the boundary is enforced, what unit it applies to, and when it is refused
What else comes with it?	what the matching languages nearly all provide too, with present and full counted separately
What am I unlikely to get?	the mirror — what few of them provide, and what none of them does
What breaks it, and where?	boundaries with no enforcement at all, then the ones that broke, by what ordinary construct, at which stage, and what restricting a route does and does not remove

Ticks are yours; marks are the study's. A checkbox you tick narrows the result. What a looked-up language records is **marked** — this one — and never ticked, because the tool selecting on your behalf is indistinguishable, once ticked, from you having chosen it. On a property the language does not have, the marked construct is what it has *instead*, and the strip is headed *recorded instead as* rather than *realised as*.

How to read it. Statements are plain by default; a badge appears only on the two kinds that ask you to do something — *worth acting on*, and *no boundary here*. Figures are set in tabular numerals so they can be compared down a column, and language and property identifiers are picked out where the tool is certain of them. It is certain from the data, never from the shape of a word: an earlier version read the `d` in a TypeScript `shapes.d.ts` as the language `d`, so identifiers are marked only where a value is exactly one.

Every statement opens onto the records it was computed from. A route record names the facility, the file and the line it was found at, the command, and what the run printed.

Three ways in. *Explore a combination* for a requirement. *Look up a studied language* for one of the 44. *Walkthrough with Unison* for a language the study never evaluated, scored from outside it with the same framework — empty until those probes are run, and it says so rather than guessing.

What it will not do

- **It does not predict your codebase.** It reports what was recorded for languages resembling yours. Holding each language out and predicting its outcomes from the rest is right 137 times in 176 — worse than assuming everything falls. The tool says so where it matters.
- **It recommends nothing.** Every remedy worth recommending is a claim this study never tested: we probed no scanner, sandbox or policy engine. Exposure is reported; what to do about it is §6.
- **It does not measure likelihood.** Whether a boundary *can* be crossed was measured. How often anyone does was not.

Running and checking it

```
open index.html          # or double-click it
python3 serve.py         # optional: serves with caching off, for editing
```

The data ships twice: as canonical JSON under data/, and as data/*.js bundles that assign it to globals. The bundles exist because a page on file:// cannot fetch a sibling file. Regenerate them whenever the JSON changes:

```
python3 ../../../../analysis/build_file_bundle.py
```

Everything here is checked, and the checks are the argument:

```
python3 ../../../../analysis/check_step10.py # 200 generated selections, every token traced
python3 ../../../../analysis/check_step11.py # the folder is complete and the bundles are
current
```

check_step10.py renders every sentence 200 selections produce and verifies that **every number, every language name and every property identifier traces to a value a query returned** — and that those queries agree, field by field, with a second implementation written independently.

The files

index.html, app.css	the page. app.js holds no knowledge about the corpus — it typesets what the templates produce and decides nothing about what is said
query.js	the queries over the facts — pure, synchronous, no prose
templates.js	the only place prose is written. No template string may contain a digit, a language name or a property id
rules.js	the five composition rules, which fire on a selection rather than on a language
provenance.js	resolves a citation to the record behind it. May not state anything: it reads fields and labels them
data/evaluation.json	layer 1 — the facts. 880 cells, 362 route records, 6,718 adjudications
data/patterns.json	layer 2 — what the study says about them, including the leave-one-out validation
data/unison.json	layer 3 — the walkthrough row, a language the study did not evaluate
serve.py	http.server with caching off